

13 — Dynamic Programming (1D)

DSA in Python — Interview Prep

Dynamic Programming (1D) — Tutorial

Covers the 10 problems solved in this folder. Every DP problem answers the same underlying question: "if I already knew the answer to every smaller version of this problem, how would I combine those answers to get the answer for the current size?" 1D DP means that state is indexed by a single number (a position, a length, an amount) rather than a pair — the topic after this one, [14_dynamic_programming_2d](#), is exactly this same idea with two indices instead of one.

The patterns

1. Rolling-pair state — only the last two answers matter (01, 02, 04, 09)

The single most common 1D DP shape here. `dp[i]` depends on only `dp[i-1]` and `dp[i-2]`, so there's no need for a full array — two rolling variables (`prev2`, `prev1`), updated each step, are enough and drop space from $O(n)$ to $O(1)$. What changes between these four problems is only *how* the two prior states combine:

- `climbing_stairs`: `prev1 + prev2` (Fibonacci — sum the ways).
- `house_robber`: `max(prev1, prev2 + n)` (choice under a can't-use-adjacent constraint).
- `min_cost_climbing_stairs`: `min(prev1 + cost, prev2 + cost)` (cheapest of two predecessor paths).
- `decode_ways`: a gated version of `climbing_stairs` — each of the two "step sizes" (one digit / two digits) only contributes if it passes a validity check first.

Recognizing "this problem's rolling-pair combination function is `+`, `max`, `min`, or gated-`+`" is often the fastest way to solve a new problem in this shape live.

2. Reducing a structural wrinkle to two calls of the simpler version (03)

`house_robber_ii`'s only difference from `02` is that the first and last houses are also adjacent (circular, not linear). Rather than deriving new circular-aware DP logic, the wraparound constraint is sidestepped entirely: any valid plan excludes house 0 or excludes the last house (never robs both), so the answer is just the better of two ordinary linear subproblems, each solved with `02`'s unchanged function. Worth remembering as a general move: a small structural complication in a problem statement doesn't always need a structural complication in the DP — sometimes it just needs the easier version called twice.

3. Full DP array — more than a constant number of predecessors matter (05, 06, 07, 08)

When `dp[i]` could depend on any of several earlier states (not just the fixed previous one or two), a rolling pair isn't enough — the full `dp` array is needed so any earlier value can be looked up.

- `coin_change`: `dp[a]` depends on `dp[a-c]` for *every* coin `c` — however many denominations there are, that many possible predecessors.
- `longest_increasing_subsequence`: `dp[i]` depends on `dp[j]` for every `j < i` with `nums[j] < nums[i]` — a data-dependent, variable number of predecessors.

- **word_break**: `dp[i]` depends on `dp[j]` for every valid split point `j` — same "try every earlier position" shape.

4. Same array, opposite loop order, different question (05 vs 06)

`coin_change` (minimum coins) and `coin_change_ii` (number of combinations) operate on the exact same kind of `dp[amount]` array, but `coin_change_ii` requires coins as the **outer** loop and amount as the **inner** loop — the opposite of what feels natural coming from 05. That ordering is what prevents counting `{1, 2}` as two separate results ("1 then 2" and "2 then 1"): processing one coin denomination completely before moving to the next means a combination is only ever built in one fixed order. Swapping the loop order back counts **permutations** instead — a genuinely different question that happens to use nearly identical code. This is the single most instructive "same-looking code, different loop order, different answer" example in the folder.

5. Reachability DP instead of enumeration (08)

`word_break` asks "can this be done" (yes/no), which is what turns what would otherwise be exponential backtracking (as in `10_recursion_backtracking/08_palindrome_partitioning`, which enumerates **every** valid partition) into polynomial DP — each `dp[i]` is computed once and reused by every later position that needs it, instead of being re-explored from scratch down every branch of a search tree.

6. Tracking two running extremes instead of one (10)

`maximum_product_subarray` looks like Kadane's algorithm (`01_arrays_strings/04_max_subarray`) with multiplication swapped in for addition, but that substitution alone is broken: a very negative running product can become the **maximum** the instant another negative number multiplies it. The fix is tracking both a running max **and** a running min at every position, since either one could flip into the new best value depending on the sign of the next element. Whenever a running Kadane's-style accumulator involves multiplication (or any operation where sign flips can invert the ranking), that's the signal to track a min alongside the max, not just the max alone.

Complexity summary

#	Problem	Time	Extra space
01	<code>climbing_stairs</code>	$O(n)$	$O(1)$
02	<code>house_robber</code>	$O(n)$	$O(1)$
03	<code>house_robber_ii</code>	$O(n)$	$O(1)$
04	<code>min_cost_climbing_stairs</code>	$O(n)$	$O(1)$
05	<code>coin_change</code>	$O(\text{amount} \cdot \text{num_coins})$	$O(\text{amount})$
06	<code>coin_change_ii</code>	$O(\text{amount} \cdot \text{num_coins})$	$O(\text{amount})$
07	<code>longest_increasing_subsequence</code>	$O(n^2)$	$O(n)$
08	<code>word_break</code>	$O(n^2)$ (n^3 with slicing cost)	$O(n)$
09	<code>decode_ways</code>	$O(n)$	$O(1)$
10	<code>maximum_product_subarray</code>	$O(n)$	$O(1)$

07's $O(n^2)$ has a well-known $O(n \log n)$ alternative (patience sorting with binary search) — worth knowing it exists and roughly how it works, but the $O(n^2)$ version here is the one to have completely solid first, since it's

what most interviewers expect derived live before asking about the optimization.

Common interview follow-ups to be ready for

- `climbing_stairs`: "what if you could climb 1, 2, or 3 steps?" — the rolling-pair becomes a rolling-triple (`prev3 + prev2 + prev1`); good to show the pattern generalizes rather than being memorized for exactly two step sizes.
- `house_robber`: "can you reconstruct *which* houses were robbed, not just the total?" — needs the full dp array (not rolling variables) plus a backtrack from the end, choosing "robbed" whenever `dp[i] != dp[i-1]`.
- `coin_change`: "what if a solution isn't guaranteed to exist?" — already handled here (`-1` return); make sure to state the `float('inf')` sentinel trick explicitly if asked to explain it.
- `coin_change_ii`: be ready to swap the loop order live and explain *in words* why it changes combinations into permutations — this is the question most likely to come up as a live follow-up here.
- `longest_increasing_subsequence`: "can you get $O(n \log n)$?" — sketch patience sorting: maintain a list of smallest tail values for increasing subsequences of each length, binary search for where each new element belongs, and the final list's length is the answer.
- `word_break`: "can you return one valid segmentation, not just yes/no?" — store a `break_point` array alongside `dp`, or switch to the backtracking version from `10_recursion_backtracking/08_palindrome_partitioning`'s shape if *all* segmentations are needed instead of just one.
- `maximum_product_subarray`: "why track a running min if the question only asks for a max?" — be ready with the sign-flip explanation as a one-liner, since this is the most commonly misunderstood part of the solution.

Self-check before moving on

You should be able to, without looking at the code:

- State which of `01/02/04/09` uses which combination function (`+`, `max`, `min`, gated `+`) from memory.
- Explain the "reduce circular to two linear calls" trick from `03` and identify when a similar reduction would apply to a new problem.
- Explain, in one sentence each, why `coin_change` needs a full array while `climbing_stairs` only needs two variables.
- Swap `coin_change_ii`'s loop order on paper and explain what the swapped version would actually be counting.
- Explain why `maximum_product_subarray` needs both a running max and a running min, with a concrete 3-element example that breaks a max-only version.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `14_dynamic_programming_2d`.

Programs

01. Climbing Stairs

01_climbing_stairs.py

Explanation

Problem: You're climbing a staircase of n steps; each move you can climb 1 or 2 steps. Return the number of distinct ways to reach the top.

Source : LeetCode 70 - Climbing Stairs

Example:

Input: $n = 5$

Output: 8

Idea: the simplest possible 1D DP, and the template for every problem in this folder: the number of ways to reach step n is the number of ways to reach step $n-1$ (then take one more step) plus the number of ways to reach step $n-2$ (then take a two-step jump) - literally the Fibonacci recurrence, arrived at independently from the problem's own structure rather than memorized. Rather than an explicit dp array, only the last two values are ever needed at once, so two rolling variables (`prev2`, `prev1`) suffice - $O(1)$ space instead of $O(n)$.

This "only the last k states matter" observation is what turns a naive $O(2^n)$ recursive branch-on-every-choice solution into a linear one-pass computation.

Code

```
def climb_stairs(n: int) -> int:
    if n <= 2:
        return n

    prev2, prev1 = 1, 2 # ways to reach step 1, ways to reach step 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2

    return prev1

if __name__ == "__main__":
    tests = [
        (1, 1),
        (2, 2),
        (3, 3),
        (5, 8),
    ]

    for i, (n, expected) in enumerate(tests, 1):
        got = climb_stairs(n)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. House Robber

02_house_robber.py

Explanation

Problem: Given the amount of money at each house along a street, find the maximum amount that can be robbed without robbing two adjacent houses.

Source : LeetCode 198 - House Robber

Example:

Input: [1, 2, 3, 1]

Output: 4 (rob house 0 and house 2: 1 + 3)

Idea: at each house, there are exactly two choices - skip it (best so far is whatever the best was up to the previous house), or rob it (this house's value plus the best achievable up to *two* houses ago, since the immediately preceding house can't also be robbed). The "best up to here" only ever depends on the best-up-to-previous and best-up-to-two-before, the same rolling-pair shape as 01's `climb_stairs`, just with `max` combining the two prior states through an adjacency constraint instead of adding them unconditionally.

Code

```
def rob(nums: list[int]) -> int:
    prev2, prev1 = 0, 0 # best up to two houses ago, best up to previous house

    for n in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + n)

    return prev1

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 1], 4),
        ([2, 7, 9, 3, 1], 12),
        ([], 0),
        ([5], 5),
        ([2, 1, 1, 2], 4),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = rob(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. House Robber II

03_house_robber_ii.py

Explanation

Problem: Same as 02, but the houses are arranged in a circle (the first and last houses are also adjacent).

Source : LeetCode 213 - House Robber II

Example:

Input: [2, 3, 2]

Output: 3 (robbing both house 0 and house 2 isn't allowed - they're adjacent in the circle - so the best is just house 1 alone)

Idea: the circular adjacency between first and last house is the only thing standing between this and 02's exact solution. Rather than writing new DP logic to handle that wraparound case directly, sidestep it: a valid robbery plan on a circle can never include *both* the first and last house, so it must fit entirely within either "all houses except the last" or "all houses except the first" - two ordinary *linear* (non-circular) subproblems, each solvable with 02's unchanged logic. The answer is just the better of those two results. This "reduce a harder variant to two calls of the easier version" move is a common way to handle a small structural wrinkle (circularity, here) without rederiving the whole recurrence.

Code

```
def rob_linear(houses: list[int]) -> int:
    prev2, prev1 = 0, 0
    for n in houses:
        prev2, prev1 = prev1, max(prev1, prev2 + n)
    return prev1

def rob_circular(nums: list[int]) -> int:
    if len(nums) == 1:
        return nums[0]
    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

if __name__ == "__main__":
    tests = [
        ([2, 3, 2], 3),
        ([1, 2, 3, 1], 4),
        ([1, 2, 3], 3),
        ([5], 5),
        ([1, 3, 1, 3, 100], 103),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = rob_circular(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Min Cost Climbing Stairs

04_min_cost_climbing_stairs.py

Explanation

Problem: Given a cost array where `cost[i]` is the cost to step on stair `i`, and you can start from step 0 or step 1 and move 1 or 2 steps at a time, find the minimum cost to reach the top (one step past the last index).

Source : LeetCode 746 - Min Cost Climbing Stairs

Example:

Input: [10, 15, 20]

Output: 15 (start at step 1, pay 15, jump 2 steps to the top)

Idea: same rolling-pair shape as 01/02, but the recurrence combines the prior two states with `min` instead of `max/+` - reaching step `i` costs whichever is cheaper: coming from step `i-1` (pay `cost[i-1]`) or from step `i-2` (pay `cost[i-2]`). The "top" is a virtual position one past the last stair, reached for free once its cheaper predecessor stair has been paid for - so the loop runs through index `n` (`len(cost)`) inclusive, not just to `n-1`.

Code

```
def min_cost_climbing_stairs(cost: list[int]) -> int:
    n = len(cost)
    prev2, prev1 = 0, 0 # cost to reach step 0, cost to reach step 1

    for i in range(2, n + 1):
        curr = min(prev1 + cost[i - 1], prev2 + cost[i - 2])
        prev2, prev1 = prev1, curr

    return prev1

if __name__ == "__main__":
    tests = [
        ([10, 15, 20], 15),
        ([1, 100, 1, 1, 1, 100, 1, 1, 100, 1], 6),
        ([0, 0, 0, 0], 0),
    ]

    for i, (cost, expected) in enumerate(tests, 1):
        got = min_cost_climbing_stairs(cost)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Coin Change

05_coin_change.py

Explanation

Problem: Given coin denominations and a target amount, return the fewest number of coins needed to make that amount (unlimited supply of each denomination), or -1 if it can't be made exactly.

Source : LeetCode 322 - Coin Change

Example:

Input: coins = [1, 2, 5], amount = 11

Output: 3 (5 + 5 + 1)

Idea: $dp[a]$ = minimum coins to make amount a , built bottom-up from $dp[0] = 0$. For every amount from 1 to target, try every coin denomination as "the last coin used" - if that coin's value fits ($c \leq a$), the best way to make a using it last is 1 (for this coin) plus the best way to make the remaining $a - c$. Take the minimum over all coin choices. Unlike 04's fixed two predecessors, each amount here can be reached from *many* possible previous amounts (one per coin denomination) - the DP array replaces the two rolling variables from earlier problems because more than a constant number of prior states matter.

Code

```
def coin_change(coins: list[int], amount: int) -> int:
    dp = [0] + [float("inf")] * amount

    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)

    return dp[amount] if dp[amount] != float("inf") else -1

if __name__ == "__main__":
    tests = [
        ([1, 2, 5], 11, 3),
        ([2], 3, -1),
        ([1], 0, 0),
        ([1, 3, 4], 6, 2),
    ]

    for i, (coins, amount, expected) in enumerate(tests, 1):
        got = coin_change(coins, amount)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Coin Change II

06_coin_change_ii.py

Explanation

Problem: Given coin denominations and a target amount, return the number of distinct *combinations* of coins that make up that amount (order doesn't matter - using a 1 then a 2 is the same combination as a 2 then a 1, so both shouldn't be counted separately).

Source : LeetCode 518 - Coin Change II

Example:

Input: amount = 5, coins = [1, 2, 5]

Output: 4 ({5}, {2,2,1}, {2,1,1,1}, {1,1,1,1,1})

Idea: same dp[a]-over-amounts idea as 05, counting instead of minimizing (dp[a] += dp[a-c] instead of dp[a] = min(...)), but the loop order is now the entire point of the problem, not a stylistic choice. Coins must be the **outer** loop, amounts the inner loop: processing one coin denomination fully (updating dp for every amount) before moving to the next coin means, by the time coin c2 is being considered, dp[a] already reflects "ways using only coins seen so far" - so a combination like {1, 2} only ever gets counted once (built in the order the coins loop visits them), never once as "1 then 2" and again as "2 then 1". Swapping the loop order (amount outer, coin inner - as in 05) instead counts **permutations**, a different question entirely; this is the single most important detail to get right in this problem, and worth being able to explain, not just recite.

Code

```
def change(amount: int, coins: list[int]) -> int:
    dp = [0] * (amount + 1)
    dp[0] = 1

    for c in coins:
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]

    return dp[amount]

if __name__ == "__main__":
    tests = [
        (5, [1, 2, 5], 4),
        (3, [2], 0),
        (10, [10], 1),
        (0, [1, 2], 1),
    ]

    for i, (amount, coins, expected) in enumerate(tests, 1):
        got = change(amount, coins)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Longest Increasing Subsequence

07_longest_increasing_subsequence.py

Explanation

Problem: Given an integer array nums, return the length of the longest strictly increasing subsequence (elements don't need to be contiguous, just in increasing order and in their original relative order).

Source : LeetCode 300 - Longest Increasing Subsequence

Example:

Input: [10, 9, 2, 5, 3, 7, 101, 18]

Output: 4 (the subsequence [2, 3, 7, 101])

Idea: $dp[i]$ = length of the longest increasing subsequence *ending exactly at index i *. For each i , check every earlier index j : if $nums[j] < nums[i]$, then i could extend whatever subsequence ends at j , giving a candidate length $dp[j] + 1$. Taking the best such candidate over all valid j (or just 1, if no earlier element is smaller - the subsequence containing just $nums[i]$ itself) gives $dp[i]$. The answer is the best $dp[i]$ across the whole array, not necessarily $dp[n-1]$, since the longest subsequence doesn't have to end at the last element. This is $O(n^2)$ - there's a well-known $O(n \log n)$ alternative using binary search (patience sorting), but the $O(n^2)$ version here is the one to have completely solid first, since it's what most interviewers expect to see derived live.

Code

```
def length_of_lis(nums: list[int]) -> int:
    if not nums:
        return 0

    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)

if __name__ == "__main__":
    tests = [
        ([10, 9, 2, 5, 3, 7, 101, 18], 4),
        ([0, 1, 0, 3, 2, 3], 4),
        ([7, 7, 7, 7], 1),
        ([], 0),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = length_of_lis(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

08. Word Break

08_word_break.py

Explanation

Problem: Given a string s and a dictionary of words, return True if s can be segmented into a sequence of one or more dictionary words (words may be reused any number of times).

Source : LeetCode 139 - Word Break

Example:

Input: s = "leetcode", word_dict = ["leet", "code"]

Output: True ("leet" + "code")

Idea: $dp[i]$ = "can the prefix $s[:i]$ be fully segmented into dictionary words?" $dp[0] = \text{True}$ (the empty prefix trivially can, with zero words) is the base case that everything else builds from. For each position i , check every earlier split point j : if $s[:j]$ is already known segmentable ($dp[j]$ is True) and the remaining piece $s[j:i]$ is itself a dictionary word, then $s[:i]$ is segmentable too. This is structurally the same "try every split point" shape as [10_recursion_backtracking/08_palindrome_partitioning](#), except here only a yes/no reachability is needed (not every possible partition enumerated), which is exactly what turns an exponential backtracking search into a polynomial DP - each $dp[i]$ is computed once and reused, instead of being re-explored from scratch down every branch.

Code

```
def word_break(s: str, word_dict: list[str]) -> bool:
    words = set(word_dict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break

    return dp[n]

if __name__ == "__main__":
    tests = [
        ("leetcode", ["leet", "code"], True),
        ("applepenapple", ["apple", "pen"], True),
        ("catsandog", ["cats", "dog", "sand", "and", "cat"], False),
        ("", ["a"], True),
    ]

    for i, (s, word_dict, expected) in enumerate(tests, 1):
        got = word_break(s, word_dict)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

09. Decode Ways

09_decode_ways.py

Explanation

Problem: A digit string can be decoded to letters via 'A' -> 1 ... 'Z' -> 26. Given a digit string s, return the number of ways it can be decoded. (A leading zero in any 1- or 2-digit group makes that

grouping invalid - '0' alone decodes to nothing.)

Source : LeetCode 91 - Decode Ways

Example:

Input: "226"

Output: 3 ("2,2,6" -> "BBF", "22,6" -> "VF", "2,26" -> "BZ")

Idea: same rolling-pair shape as 01's climbing stairs - the number of ways to decode the prefix ending at position i is built from the number of ways ending at $i-1$ (if the single digit at i is valid, i.e. not '0') plus the number of ways ending at $i-2$ (if the *two* digits ending at i form a valid 10-26 group). This is genuinely climbing stairs with an extra validity condition gating each of the two "step sizes" - worth noticing the structural similarity rather than treating this as an unrelated new problem. The two edge conditions (a lone '0' is always invalid; a two-digit group must be 10-26, not any two digits) are what most solutions get wrong first try - they're the entire difficulty of this problem, not the DP shape.

Code

```
def num_decodings(s: str) -> int:
    if not s or s[0] == "0":
        return 0

    n = len(s)
    prev2, prev1 = 1, 1 # ways to decode "" (empty), ways to decode s[:1]

    for i in range(2, n + 1):
        curr = 0
        one_digit = int(s[i - 1])
        two_digit = int(s[i - 2:i])

        if one_digit >= 1:
            curr += prev1
        if 10 <= two_digit <= 26:
            curr += prev2

        prev2, prev1 = prev1, curr

    return prev1

if __name__ == "__main__":
    tests = [
        ("12", 2),
        ("226", 3),
        ("06", 0),
        ("0", 0),
        ("10", 1),
        ("27", 1),
        ("100", 0),
    ]

    for i, (s, expected) in enumerate(tests, 1):
        got = num_decodings(s)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

10. Maximum Product Subarray

10_maximum_product_subarray.py

Explanation

Problem: Given an integer array `nums`, find the contiguous subarray with the largest product, and return that product.

Source : LeetCode 152 - Maximum Product Subarray

Example:

Input: [2, 3, -2, 4]

Output: 6 (subarray [2, 3])

Idea: looks like [01_arrays_strings/04_max_subarray](#) (Kadane's algorithm) with multiplication instead of addition, but that substitution alone is broken - a single running "best product ending here" isn't enough, because a *very negative* running product can become the new *maximum* the instant it's multiplied by another negative number. The fix: track both a running max and a running min product ending at each position. At every step, the new max is the best of (current element alone, running max * current element, running min * current element) - that third option is what correctly captures "two negatives about to cancel out into a big positive." The running min needs the same three-way comparison, symmetrically, since a small-but-currently-minimal product could itself flip into the new maximum on the *next* step if another negative shows up.

Code

```
def max_product(nums: list[int]) -> int:
    result = nums[0]
    curr_max = curr_min = nums[0]

    for n in nums[1:]:
        candidates = (n, curr_max * n, curr_min * n)
        curr_max = max(candidates)
        curr_min = min(candidates)
        result = max(result, curr_max)

    return result

if __name__ == "__main__":
    tests = [
        ([2, 3, -2, 4], 6),
        ([-2, 0, -1], 0),
        ([-2, 3, -4], 24),
        ([5], 5),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = max_product(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```